JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class ObjectInputStream.GetField

java.lang.Object
    java.io.ObjectInputStream.GetField

Enclosing class:

ObjectInputStream

```
public abstract static class ObjectInputStream.GetField
extends Object
```

Provide access to the persistent fields read from the input stream.

Since:

1.1

Constructor Summary

Constructors

Constructor	Description
`GetField()`	Constructor for subclasses to call.

Method Summary

All Methods | **Instance Methods**

Modifier and Type	Method	Description
`abstract boolean`	`defaulted(String name)`	Return true if the named field is defaulted and has no value in this stream.
`abstract boolean`	`get(String name, boolean val)`	Get the value of the named boolean field from the persistent field.
`abstract byte`	`get(String name, byte val)`	Get the value of the named byte field from the persistent field.
`abstract char`	`get(String name, char val)`	Get the value of the named char field from the persistent field.
`abstract double`	`get(String name, double val)`	

		Get the value of the named double field from the persistent field.
abstract float	**get**(String name, float val)	Get the value of the named float field from the persistent field.
abstract int	**get**(String name, int val)	Get the value of the named int field from the persistent field.
abstract long	**get**(String name, long val)	Get the value of the named long field from the persistent field.
abstract short	**get**(String name, short val)	Get the value of the named short field from the persistent field.
abstract Object	**get**(String name, Object val)	Get the value of the named Object field from the persistent field.
abstract ObjectStreamClass	**getObjectStreamClass**()	Get the ObjectStreamClass that describes the fields in the stream.

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Constructor Details

GetField

```
public GetField()
```

Constructor for subclasses to call.

Method Details

getObjectStreamClass

```
public abstract ObjectStreamClass getObjectStreamClass()
```

Get the ObjectStreamClass that describes the fields in the stream.

Returns:

the descriptor class that describes the serializable fields

defaulted

```
public abstract boolean defaulted(String name)
                           throws IOException
```

Return true if the named field is defaulted and has no value in this stream.

Parameters:

`name` - the name of the field

Returns:

true, if and only if the named field is defaulted

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`IllegalArgumentException` - if `name` does not correspond to a serializable field

get

```
public abstract boolean get(String name,
                            boolean val)
                     throws IOException
```

Get the value of the named boolean field from the persistent field.

Parameters:

`name` - the name of the field

`val` - the default value to use if `name` does not have a value

Returns:

the value of the named `boolean` field

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

IllegalArgumentException - if type of name is not serializable or if the field type is incorrect

get

```
public abstract byte get(String name,
                         byte val)
                  throws IOException
```

Get the value of the named byte field from the persistent field.

Parameters:

name - the name of the field

val - the default value to use if name does not have a value

Returns:

the value of the named byte field

Throws:

IOException - if there are I/O errors while reading from the underlying InputStream

IllegalArgumentException - if type of name is not serializable or if the field type is incorrect

get

```
public abstract char get(String name,
                         char val)
                  throws IOException
```

Get the value of the named char field from the persistent field.

Parameters:

name - the name of the field

val - the default value to use if name does not have a value

Returns:

the value of the named char field

Throws:

IOException - if there are I/O errors while reading from the underlying InputStream

IllegalArgumentException - if type of name is not serializable or if the field type is incorrect

get

```
public abstract short get(String name,
                          short val)
                   throws IOException
```

Get the value of the named short field from the persistent field.

Parameters:

`name` - the name of the field

`val` - the default value to use if `name` does not have a value

Returns:

the value of the named `short` field

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`IllegalArgumentException` - if type of `name` is not serializable or if the field type is incorrect

get

```
public abstract int get(String name,
                        int val)
                 throws IOException
```

Get the value of the named int field from the persistent field.

Parameters:

`name` - the name of the field

`val` - the default value to use if `name` does not have a value

Returns:

the value of the named `int` field

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`IllegalArgumentException` - if type of `name` is not serializable or if the field type is incorrect

get

```
public abstract long get(String name,
                         long val)
                  throws IOException
```

Get the value of the named long field from the persistent field.

Parameters:

`name` - the name of the field

`val` - the default value to use if `name` does not have a value

Returns:

the value of the named `long` field

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`IllegalArgumentException` - if type of `name` is not serializable or if the field type is incorrect

get

```
public abstract float get(String name,
                          float val)
                   throws IOException
```

Get the value of the named float field from the persistent field.

Parameters:

`name` - the name of the field

`val` - the default value to use if `name` does not have a value

Returns:

the value of the named `float` field

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`IllegalArgumentException` - if type of `name` is not serializable or if the field type is incorrect

get

```
public abstract double get(String name,
                           double val)
                    throws IOException
```

Get the value of the named double field from the persistent field.

Parameters:

`name` - the name of the field

`val` - the default value to use if `name` does not have a value

Returns:

the value of the named `double` field

Throws:

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`IllegalArgumentException` - if type of `name` is not serializable or if the field type is incorrect

get

```
public abstract Object get(String name,
                           Object val)
                    throws IOException,
                           ClassNotFoundException
```

Get the value of the named Object field from the persistent field.

Parameters:

`name` - the name of the field

`val` - the default value to use if `name` does not have a value

Returns:

the value of the named `Object` field

Throws:

`ClassNotFoundException` - Class of a serialized object cannot be found.

`IOException` - if there are I/O errors while reading from the underlying `InputStream`

`IllegalArgumentException` - if type of `name` is not serializable or if the field type is incorrect

---

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.